

ADR-0007 — Presentational/container split with a hard cap on event-forwarding depth

ACCEPTED Date: 2026-09-21 · Deciders: Nicolà Widmer

Context

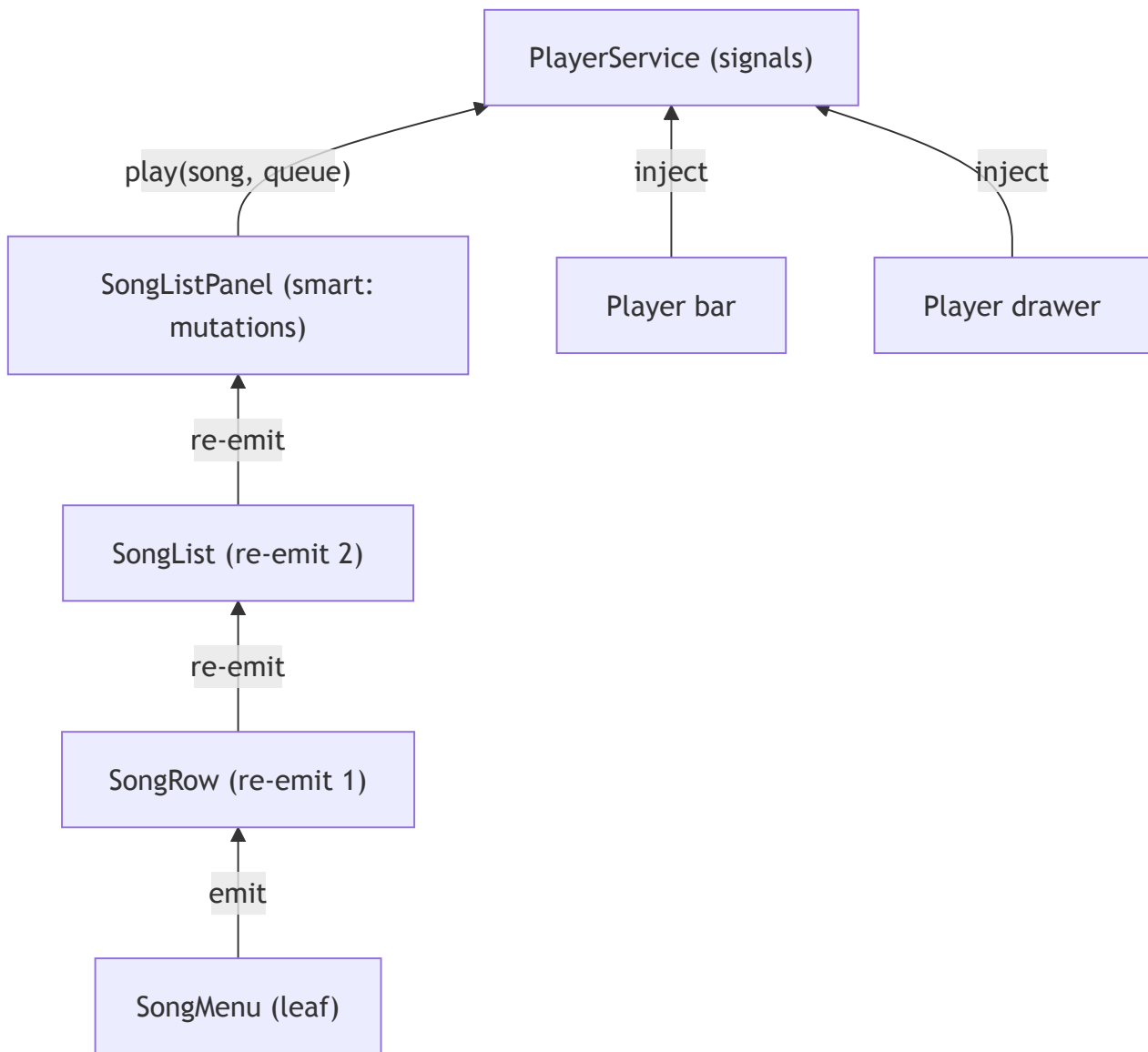
Clicks in small components (menu, row, transport buttons) must reach the component that owns the mutation. Re-emitting `output()`s through many layers is error-prone: a missed re-emit fails silently. Playback state is needed by the bar and the drawer at the same time.

Options

1. **Presentational leaves emit; at most two re-emits; one smart component owns the mutation; shared client state in a service**
2. Pass everything through inputs/outputs, any depth
3. One global store (NgRx)
4. Leaves inject the container's logic directly
5. Cache the visible list in `PlayerService` (or a `QueueService`) so rows can call `play(song)` themselves
6. A panel-scoped context service (`providers: [SongListActions]` on the panel) that rows and menus inject

Decision

Option 1.



- Leaves use only `input()` / `output()`. No queries, no route params.
- Smart components: `SongListPanel`, `PlaylistsPage`, the edit/create dialogs.
- More than two re-emits → move the smart component closer or use a service.
- A service is only for state that several mounted components show at once.

Why the chain has three hops

The runtime view shows `SongRow` → `SongList` → `SongListPanel` → `PlayerService`. The two middle hops add nothing: each is one template binding (`((play)="play.emit($event)")`). The chain exists because only the panel has the context the event needs:

- **Queue.** `PlayerService.play(song, queue)` needs the list in display order. A row knows one song. The panel owns `rows()` and derives the queue from it.
- **Meaning depends on where the list is.** In the library, `delete` deletes the song. In a playlist, it removes the entry. `edit` opens a dialog the panel owns. A row can't decide this without knowing where it is mounted.
- **One path for all events.** `play`, `edit`, `delete`, `move` and `addToPlaylist` all travel the same way. If rows injected a service for `play` only, there would be two mechanisms for the same kind of event.

`PlayerService` keeps only the *playing* queue, copied at click time. That way, opening another page doesn't change which song plays next.

Keeping components small

Only the panel grows, and that's intended: it owns what happens after a click. If it gets too large, move its logic into a plain `@Injectable()` provided on the panel and injected *only by the panel*. The panel then only wires things up, and leaves still use only `input()` / `output()`.

Consequences

- Good: an event's owner is at most three components away.
- Good: leaves are reusable and easy to test.
- Good: playback has one home.
- Bad: the two-hop rule is checked in review, not by a linter.
- Bad: overusing services would bring back hidden coupling.
- Rejected NgRx: too heavy for three routes.
- Rejected injecting into leaves: they could no longer be reused, and a row lacks the queue and library/playlist context anyway.
- Rejected caching the visible list in a root service: whichever panel mounted last overwrites it, so "next" can jump into a list the user left. Fixing this needs a second "playing queue" field and register/unregister on every panel. Two panels on screen at once would conflict without any error. A separate `QueueService` has the same problems. It also always goes together with `PlayerService`, because `next()`, `previous()` and `ended` need both.
- Rejected a panel-scoped context service for now: it removes the forwarding and scopes cleanly per panel. But leaves would then depend on a provider they can't see, a missing provider is a runtime DI error instead of a compile error, and every leaf test needs the provider. Worth revisiting if the tree gets deeper than the two-re-emit cap.